

Finance-Dashboard-system API Technical Documentation

1. Executive Summary

The FinFlow API is a production-grade RESTful backend system engineered to serve as the core engine for a comprehensive financial dashboard. Developed as an internship assessment, the system facilitates secure transaction management, real-time financial analytics, and granular user administration. The primary objective was to build a robust foundation that ensures data integrity and high availability while providing developers with a clean, extensible codebase.

The architecture prioritizes security and performance, leveraging a dual-token JWT authentication mechanism coupled with a Redis-backed session management layer. By implementing multi-tier Role-Based Access Control (RBAC), the system ensures that sensitive financial operations are restricted to authorized personnel, ranging from read-only viewers to administrative overseers.

Technically, the solution is built on a Node.js and Express.js stack, utilizing MongoDB Atlas for cloud-native persistence and Redis for high-speed caching and security operations. The final product demonstrates a mastery of modern backend paradigms, including strict input validation, centralized error handling, and a defensive security posture designed to withstand common web vulnerabilities.

2. System Architecture

The FinFlow API utilizes a **Modular Monolith** architecture. This design choice provides a balance between the simplicity of a single deployment unit and the logical separation required for long-term maintainability. By organizing code into distinct modules (Auth, Records, Users, Dashboard), the system avoids the "spaghetti code" common in early-stage monoliths.

Layered Pattern Implementation

To enforce the separation of concerns, each module follows a strict four-layer hierarchy:

- **Route Layer:** Defines API endpoints and attaches specific middleware (e.g., Auth, RBAC, Validation).
- **Controller Layer:** Orchestrates the request-response lifecycle. It extracts parameters, calls appropriate services, and returns standardized JSON responses.
- **Service Layer:** Contains the core business logic. This layer remains agnostic of the HTTP protocol and the database implementation, ensuring that logic can be tested in isolation.
- **Repository Layer:** Handles all direct interactions with the MongoDB database. This abstraction allows the underlying data source to be swapped or modified with minimal impact on business logic.

3. Database Design

The system employs MongoDB Atlas as a managed NoSQL solution, allowing for a flexible schema that can accommodate evolving financial data categories without expensive migrations.

User Schema

The **User** collection manages identity and access levels:

- **name:** String for user identification.
- **email:** Validated unique string for communication and login.
- **password:** Bcrypt-hashed string (never stored in plain text).
- **role:** Enum (Viewer, Analyst, Admin).
- **isDeleted:** Boolean for soft-delete implementation to maintain audit trails.

Record Schema

The **Record** collection stores granular financial transactions:

- **userId:** Reference to the owning user.

- **amount**: Number — stores transaction value, must be positive.
- **type**: Enum (Income, Expense).
- **category**: String (e.g., Salary, Rent, Utilities).
- **date**: Date object with indexing for optimized range queries.
- **notes**: Optional text field for transaction notes (max 500 characters).

4. API Design Philosophy

The API is designed following strict **RESTful principles**, ensuring that resources are identified by URLs and manipulated using standard HTTP verbs (GET, POST, PUT, DELETE).

Consistent Response Format

All endpoints return a standardized JSON structure to ensure predictable integration for frontend consumers:

- **success**: Boolean indicating the outcome.
- **data**: The requested resource or payload (null on error).
- **error**: An object containing a machine-readable code and a human-readable message.
- **meta**: Pagination details (e.g., totalPages, currentPage, limit) for list endpoints.

Advanced Querying

The Financial Records API supports server-side filtering, sorting, and pagination. Users can filter results by transaction type, category, and date ranges, preventing over-fetching and reducing client-side processing overhead.

5. Security Implementation

Security is integrated as a cross-cutting concern throughout the application pipeline.

JWT Strategy and Redis

The system employs a dual-token strategy:

- **Access Token:** Short-lived (15 min) JWT for authenticating requests.
- **Refresh Token:** Long-lived (7 days) JWT used to obtain new access tokens.
- **Token Blacklisting:** Upon logout, tokens are added to a Redis-based blacklist. Every request checks this high-speed cache to ensure the session has not been revoked, effectively mitigating the "zombie token" issue common in stateless JWT implementations.

Role-Based Access Control (RBAC)

Role	Permissions
Viewer	Read-only access to dashboard and personal records.
Analyst	Read-only access to financial records and analytical summaries (category-wise totals and monthly trends)
Admin	Full system access, including user management and role assignment.

Middleware Chain

Every request passes through a defensive middleware chain:

- **Helmet:** Sets secure HTTP headers to prevent clickjacking and XSS.
- **CORS:** Restricts API access to authorized domains.
- **Rate Limiting:** Implemented via express-rate-limit middleware, capping 100 requests per IP per 15 minutes to prevent brute-force attacks.
- **Joi Validation:** Validates request bodies against strict schemas before they reach the controller, ensuring "fail-fast" behavior.

6. Key Technical Decisions and Why

Choice of Node.js and Express

The non-blocking I/O model of Node.js is ideal for an API handling many concurrent I/O-bound operations (database queries, Redis checks). Express provides the flexibility to build a custom middleware pipeline without the overhead of heavy frameworks.

Repository Pattern Implementation

While many Express tutorials put database logic in controllers, this project implements the Repository pattern to decouple business logic from Mongoose-specific syntax. This facilitates easier unit testing using mocks.

Soft Delete for Users

Rather than using a hard `DELETE` operation, the system toggles an `isDeleted` flag. This preserves referential integrity for financial records and allows for administrative recovery if necessary.

7. Challenges and Solutions

Challenge: Efficient Dashboard Aggregations

Calculating monthly trends and category-wise totals across thousands of records can be resource-intensive.

Solution: Utilized MongoDB's **Aggregation Framework**. By building multi-stage pipelines (e.g., `$match`, `$group`, `$sort`), complex summaries are computed entirely on the database server, returning only the final result to the API.

Challenge: Handling Global Errors

Maintaining clean code while handling various error states (Validation, Auth, DB) can lead to repetitive try-catch blocks.

Solution: Developed a **Centralized Error Handler**. Custom error classes (e.g., `AppError`, `ValidationError`) allow the service layer to throw specific exceptions that the global middleware catches and formats into the standard response structure.

8. Future Improvements

While the FinFlow API is feature-complete for an assessment, several enhancements would elevate it to production-ready status:

- **Microservices Transition:** Leveraging the "Strangler Fig" pattern to decouple the "Dashboard Summary" engine into a separate service if aggregation loads exceed a certain threshold.
- **Real-time Updates:** Integration of WebSockets (Socket.io) to push notification alerts when high-value expenses are recorded or when budgets are exceeded.
- **Automated Testing:** Implementation of a full test suite using Jest and Supertest to ensure 90%+ code coverage.
- **Infrastructure as Code:** Transitioning from manual deployment to Kubernetes (K8s) for automated scaling and self-healing in a production cluster.